ADAMA SCIENCE AND TECHNOLOGY UNIVERSITY

Department of Computer Science and Engineering

Introduction to Natural Language Processing
Code: CSEg5306

Comparative Analysis of Decoder-Only Architectures for Abstractive Text Summarization

Prepared By:

Student Name	ID Number
Abdulaziz Isa	UGR/25624/14
Baysa Dida	UGR/25935/14
Samuel Geremew	UGR/25300/14
Segni Girma	UGR/25327/14
Tariku Tamene	UGR/25598/14
Yohannes Gzachew	UGR/25637/14

Date: April 20, 2026

Abstract

This detailed documentation describes the engineering, architecture, and theoretical foundations of an abstractive text summarization system that uses the GPT-2 (Generative Pre-trained Transformer 2) framework. By combining ideas from the foundational paper by Vaswani et al. (2017) and modern interactive visualizer tools, such as the Polo Club Transformer Explainer, this study compares a model trained from scratch (with a randomized start) and a model that uses large-scale pre-trained weights fine-tuned on task-specific data. We use the Kaggle CNN/DailyMail dataset for our main goal: to measure the "pre-training advantage" and assess the effectiveness of causal self-attention in complex language tasks under strict computational limits. We break down the Query-Key-Value (QKV) attention matrix operations, the importance of pre-layer normalization, token embeddings, and autoregressive generation methods. The final metrics highlight the specific factors that influence generative fluency and syntactic coherence in decoder-only architectures.

Contents

1 Introduction **3**

1.1 Domain Context and Paradigm Shift 3

1.2 Research Objectives 3

1.3 Key Research Questions 4

2 Architecture: The Decoder-Only Foundation **4**

2.1 Evolution from Encoder-Decoder 4

2.2 Internal Data Flow 4

2.3 Transformer Block Configuration 5

3 Input Representation: Embeddings and Spatial Awareness **5**

3.1 Token Embeddings (W_e) 5

3.2 Learned Positional Encodings (W_p) 6

4 Mathematical Formulation of Self-Attention **6**

4.1 The Query, Key, Value Paradigm 7

4.2 Scaled Dot-Product Attention 7

4.3 Multi-Head Attention (MHA) 7

4.4 Causal Masking: The Autoregressive Constraint 8

5 Feed-Forward Networks and Layer Normalization **8**

5.1 Position-wise Feed-Forward Network 8

5.2 Pre-Layer Normalization (Pre-LN) 9

6 Data Collection and Preprocessing **9**

6.1 Dataset Selection: CNN/DailyMail (Kaggle Edition) 9

6.2 Cleaning and Normalization Protocol 10

6.3 Data Transformation: The Prefix Strategy 11

7 Methodology and Training Dynamics **11**

7.1 Masked Loss Computation 11

7.2 Training Protocols 12
7.2.1 Protocol A: De Novo (From Scratch) 12
7.2.2 Protocol B: Pretrained Fine-Tuning 12
7.3 Computational Complexity and Hardware Optimization 12

8 Inference: Autoregressive Decoding Strategies 13
8.1 Temperature Scaling 13
8.2 Truncation Sampling: Top-K and Top-P 13

9 Evaluation Metrics and Expected Results 14
9.1 Quantitative Benchmarks (ROUGE) 14
9.2 Hypothesized Performance Delta 14

10 Discussion and Limitations 14
10.1 Linguistic Pathologies 14
10.2 Hardware and Domain Restrictions 15

11 Conclusion 15

A Technical Appendices 17
A.1 Project Architecture and Codebase Structure 17
A.2 Algorithm: Autoregressive Inference Generation Loop 17

1 Introduction

1.1 Domain Context and Paradigm Shift

Abstractive summarization is a complex task in Natural Language Processing (NLP). It involves combining information from different sources to create new, grammatically correct sentences. Unlike extractive methods, which pick and piece together sentences from the original text, abstractive systems need to understand language rules, the meaning of documents, and broader contextual themes to accurately rephrase information.

Prior to 2017, sequence-to-sequence modeling was dominated by Recurrent Neural Networks (RNNs), Long Short-Term Memory networks (LSTMs), and Gated Recurrent Units (GRUs). While effective for short strings, these architectures suffered from sequential bottlenecking. Tokens had to be processed iteratively $(t_1, t_2, \ldots, t_n)$, which restricted hardware parallelization and caused long-range dependencies to degrade due to the vanishing gradient problem. The introduction of the Transformer architecture by Vaswani et al. in *Attention Is All You Need* fundamentally shifted this paradigm. By discarding recurrence entirely and relying strictly on an attention mechanism, the Transformer allowed for global dependency tracking across sequence lengths in $O(1)$ sequential operations.

This project uses the GPT-2 decoder-only offshoot of the Transformer architecture to look at abstractive summarisation not as a translation task but as an autoregressive conditional language modelling problem.

1.2 Research Objectives

This study isolates the "pre-training advantage" by executing a strict side-by-side comparison between two identical architectural configurations:

1. **De Novo (Scratch) Protocol:** Initialized with randomized weights, possessing zero initial linguistic prior.
2. **Pre-trained Fine-Tuning Protocol:** Utilizing weights established through large-scale pre-training on general-purpose corpora (e.g., the WebText dataset).

The study aims to ascertain whether architectural design, in the absence of prior familiarity with extensive language distributions, is adequate for mastering the intricate mapping of document condensation under stringent resource limitations.

1.3 Key Research Questions

- **Cognitive Learning:** To what extent does a decoder-only architecture independently develop summarization logic and English syntax when initialized with zero prior knowledge?
- **Data Efficiency:** What is the quantifiable performance delta (measured via ROUGE metrics) when the task-specific training dataset is strictly limited?
- **Structural Influence:** How does the causal self-attention mechanism manage the structural relationship between long-form source articles and generated summaries within a single, unified sequence vector?

2 Architecture: The Decoder-Only Foundation

2.1 Evolution from Encoder-Decoder

The initial Transformer, introduced by Vaswani et al. (2017), featured a dual-stack architecture comprising an Encoder (to convert continuous inputs into an intermediate representation) and a Decoder (to produce outputs while monitoring the encoder's state through cross-attention).

The Generative Pre-trained Transformer (GPT) series diverges from this by utilizing a **decoder-only** configuration. In this unified architecture, the model processes the source text (the article) and the target text (the summary) as a single, continuous vector. This eliminates the need for encoder-decoder cross-attention layers, relying strictly on masked self-attention to manage dependencies across the entire sequence.

2.2 Internal Data Flow

As visualized in modern interactive Transformer explainers, the lifecycle of data within the decoder architecture follows a strict, unidirectional mathematical flow:

1. Raw text is fragmented into discrete integers via Byte Pair Encoding (BPE).
2. Integers are projected into continuous high-dimensional space via an Embedding matrix and merged with Spatial Positional Encodings.
3. The continuous vector traverses N identical Transformer Blocks, updating token contexts at each layer.

4. The final contextualized vector is projected back to the vocabulary dimension to form raw logits.

5. Logits are converted to a probability distribution via Softmax for autoregressive next-token sampling.

2.3 Transformer Block Configuration

Each transformer block contains two primary sub-layers:

- **Masked Multi-Head Self-Attention (MHA):** Extracts distinct linguistic features simultaneously.
- **Position-wise Feed-Forward Network (FFN):** Expands the representation to capture non-linear combinations and factual associations.

In our implementation, the **Pretrained Variant** utilizes the standard 12-layer stack (124M parameters). To accommodate training efficiency under extreme resource constraints, the **Scratch Variant** is dynamically configured with 6 to 8 stacked transformer blocks. Both versions maintain a consistent hidden dimensionality (d_{model}) of 768.

3 Input Representation: Embeddings and Spatial Awareness

Because the transformer architecture lacks inherent recurrence or convolution, it is fundamentally permutation-invariant. Without positional data, the phrase "Dog bites man" is mathematically indistinguishable from "Man bites dog". Sequence order must be explicitly injected at the input stage.

3.1 Token Embeddings (W_e)

The model uses Byte Pair Encoding (BPE) to keep track of a vocabulary of exactly 50,257 subword units. BPE bridges the gap between character-level and word-level tokenization, allowing the model to handle rare technical vocabulary and proper nouns by breaking them down into recognizable subword chunks.

Each discrete integer token x_i is mapped to a continuous high-dimensional vector space:

$$E_{token} = x_i W_e \tag{1}$$

where $W_e \in \mathbb{R}^{|V| \times d_{model}}$ is a learnable embedding matrix. This matrix acts as a semantic lookup table. In this high-dimensional manifold, words with similar linguistic contexts (e.g., "king" and "queen") cluster near each other based on cosine similarity.

3.2 Learned Positional Encodings (W_p)

In the original *Attention Is All You Need* paper, Vaswani et al. utilized fixed, deterministic sinusoidal encodings:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \tag{2}$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \tag{3}$$

This approach theoretically allowed models to extrapolate to sequence lengths longer than those seen during training. However, empirical testing in subsequent years demonstrated that absolute learned embeddings often yielded superior empirical results on fixed-length contexts.

Therefore, GPT-2 deviates by utilizing absolute **learned positional embeddings**. A dedicated embedding matrix $W_p \in \mathbb{R}^{L \times d_{model}}$ is initialized and optimized during training, where L is the maximum context window (1024 tokens).

The final input representation h_0 for a token at spatial position i is the element-wise summation:

$$h_0 = W_e(x_i) + W_p(i) \tag{4}$$

Constraint Logic: This summation forces the model to synthesize semantic identity with spatial context seamlessly. It allows the attention mechanism to distinguish between the token "inflation" occurring in the lead paragraph of the article versus the identical token "inflation" appearing in the generated summary.

4 Mathematical Formulation of Self-Attention

Self-attention is the core cognitive engine of the Transformer. It allows a token to dynamically update its own representation by aggregating information from the surrounding sequence context.

4.1 The Query, Key, Value Paradigm

To execute self-attention, the input vector sequence $X \in \mathbb{R}^{L \times d_{model}}$ is multiplied by three separate learned weight matrices (W^Q, W^K, W^V) to create three new vectors for each token in the sequence. Drawing on database retrieval analogies:

- **Query ($Q = XW^Q$):** "What information am I looking for?" (e.g., A pronoun seeking its noun antecedent).
- **Key ($K = XW^K$):** "What information do I contain?" (The semantic identifiers of the current token).
- **Value ($V = XW^V$):** "If my Key matches your Query, here is the actual informational payload I will transmit to you."

4.2 Scaled Dot-Product Attention

The attention score representing the relevance between all tokens is computed by taking the dot product of the Query matrix and the transposed Key matrix. This results in an $L \times L$ attention correlation matrix:

$$\text{Raw Score} = QK^T \tag{5}$$

Vaswani et al. observed that for large values of the key dimension (d_k), the dot products grow astronomically large. This pushes the subsequent softmax function into regions with extremely small gradients, severely stalling training. To counteract this, the score is scaled down by the inverse square root of the key dimension ($\frac{1}{\sqrt{d_k}}$):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{6}$$

The Softmax function normalizes the scores across the sequence dimension so they sum exactly to 1. This probability distribution is then matrix-multiplied by the Value matrix (V), resulting in a contextually enriched vector representation for every token.

4.3 Multi-Head Attention (MHA)

Rather than performing a single, monolithic attention function, the architecture projects the queries, keys, and values h times ($h = 12$ parallel heads for GPT-2) into lower-

dimensional subspaces where $d_k = d_v = d_{model}/h = 64$.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{7}$$

where each head is calculated as:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \tag{8}$$

This architecture allows different heads to specialize in modeling diverse linguistic phenomena. For example, Head 1 may track grammatical subject-verb agreements, Head 4 may track coreference resolution (he → John), and Head 9 may track global thematic keywords.

4.4 Causal Masking: The Autoregressive Constraint

In a decoder-only configuration, the model is trained exclusively via Next Token Prediction. To preserve the integrity of the autoregressive process, the model is strictly forbidden from "cheating" by looking at future tokens.

To mathematically enforce this, an upper-triangular Causal Mask matrix M filled with $-\infty$ is added to the scaled dot product prior to the softmax operation:

$$M_{ij} = \begin{cases} 0 & \text{if } i \geq j \\ -\infty & \text{if } i < j \end{cases} \tag{9}$$

$$\text{Masked Attention} = \text{softmax}\left(\frac{QK^T + M}{\sqrt{d_k}}\right)V \tag{10}$$

The Softmax function guarantees that $e^{-\infty} = 0$. Therefore, attention scores for all future positions are forced to exactly zero. When applied to our abstractive summarization task, the causal mask allows the generated summary tokens to attend backward to the entirety of the source article, but blinds them to the remainder of the summary that has not yet been sequentially generated.

5 Feed-Forward Networks and Layer Normalization

5.1 Position-wise Feed-Forward Network

Following the MHA sub-layer, each token vector passes through a fully connected Feed-Forward Network (FFN). This network is applied independently and identically to each

position in the sequence. It consists of two linear transformations with a non-linear activation function situated between them.

While the original Transformer utilized a ReLU activation, GPT-2 utilizes the Gaussian Error Linear Unit (GELU), which smooths the gradient response around zero, preventing "dead neurons":

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2 \tag{11}$$

In this block, the vector dimensionality expands drastically from $d_{model} = 768$ to an inner capacity of 3072, before projecting back to 768. This expansive FFN acts as the model's primary memory bank, storing facts, syntax rules, and associations learned during pre-training.

5.2 Pre-Layer Normalization (Pre-LN)

A vital architectural modification GPT-2 implements over the original Vaswani Transformer is the re-routing of Layer Normalization.

In the original Transformer (Post-LN), normalization occurred *after* the residual connection addition. For very deep networks, this often led to highly unstable gradients at initialization. GPT-2 employs **Pre-LN**, applying the layer normalization *before* the attention and feed-forward sub-blocks.

Consequently, the residual connection completely bypasses the normalization block, creating a pristine, unobstructed gradient highway from the final output layer all the way back to the embedding layer:

$$x' = x + \text{MHA}(\text{LayerNorm}(x)) \tag{12}$$

$$x'' = x' + \text{FFN}(\text{LayerNorm}(x')) \tag{13}$$

This enhances training stability significantly, allowing the Scratch model to converge much faster without complex learning rate warm-up schedules.

6 Data Collection and Preprocessing

6.1 Dataset Selection: CNN/DailyMail (Kaggle Edition)

The dataset utilized for this study is the **Newspaper Text Summarization (CNN/DailyMail)** corpus, specifically the curated version hosted on Kaggle by author Gowrishankar P.

Originally constructed by Hermann et al. (2015) for machine reading comprehension, this dataset has evolved into the definitive industry standard benchmark for abstractive summarization architectures.

It contains English-language news articles written by journalists at CNN and the Daily Mail, paired with multi-sentence, bulleted abstractive highlights. The Kaggle distribution provides a highly structured CSV format containing three primary variables: `id` (a unique string identifier), `article` (the long-form journalistic source text), and `highlights` (the target summary).

To ensure robust model evaluation and prevent data leakage, the corpus is pre-split into three standard partitions:

- **Training Set:** 287,113 article-summary pairs utilized for iterative model weight optimization and gradient descent.
- **Validation Set:** 13,368 pairs utilized for dynamic hyperparameter tuning and preventing model overfitting during training.
- **Testing Set:** 11,490 pairs reserved exclusively for the final zero-shot and fine-tuned ROUGE evaluation.

This massive scale ensures the model is exposed to a vast variance of journalistic prose—ranging from global political reporting to sports summaries—ensuring robust semantic generalization.

6.2 Cleaning and Normalization Protocol

- **Artifact Removal:** Recurring non-semantic noise that provides no summarization value (e.g., author bylines, publication timestamps, agency tags like "REUTERS") are systematically stripped using regular expressions.
- **Standardization:** Text is strictly normalized to UTF-8. Curly quotes, em-dashes, and special ligatures are flattened to standard ASCII equivalents to prevent unnecessary fragmentation in the BPE vocabulary.
- **Filtering:** Data pairs with extreme length anomalies (e.g., where the summary length is $< 10\%$ of the source article) are purged to ensure the dataset maintains a consistent signal-to-noise ratio.

6.3 Data Transformation: The Prefix Strategy

In a traditional sequence-to-sequence model, the data is inherently separated: source to encoder, target to decoder. Because GPT-2 is a unified decoder, the data must be formatted as a single continuous vector. We achieve task orientation utilizing a "Prefix Strategy":

```
<|startoftext|> [Article Body] \n\nTL;DR:\n [Summary Body] <|endoftext|>
```

The injection of the literal separator `TL;DR:` acts as a behavioral trigger. During training, the model mathematically learns that the probability distribution of tokens radically shifts from "expansive reporting" to "condensed highlighting" immediately following this specific sequence of characters.

7 Methodology and Training Dynamics

The training phase formulates summarization purely as a language modeling task, optimizing the cross-entropy loss over the predicted token probabilities.

7.1 Masked Loss Computation

During the forward pass, the model yields logit predictions for the entire 1024-token sequence. The Cross-Entropy Loss function is defined as:

$$L_{CE} = -\sum_{c=1}^{|V|} y_c \log(p_c) \tag{14}$$

where y is the true one-hot target distribution and p is the model's softmax probability vector.

Critical Optimization: We deliberately "mask" the loss calculated over the Article section. The optimizer only accumulates gradients and penalizes the model for incorrect predictions that occur *after* the `TL;DR:` separator. This focuses the computational power entirely on improving summarization generation, rather than teaching the model to memorize the news article text itself.

7.2 Training Protocols

7.2.1 Protocol A: De Novo (From Scratch)

The "Scratch" variant is initialized using a truncated normal distribution ($\mu = 0, \sigma = 0.02$). This model enters the training phase with maximum entropy (zero linguistic knowledge). It must simultaneously decode English vocabulary mappings, syntax structures, and document condensation logic. Due to the high initial variance, the learning rate is set aggressively at 1×10^{-4} coupled with a 100-step linear warmup.

7.2.2 Protocol B: Pretrained Fine-Tuning

This variant utilizes the precise weights of the GPT-2 124M model, which was originally trained on 40GB of diverse internet text (WebText). It enters the summarization task already possessing a deep linguistic prior. To prevent *catastrophic forgetting* of this invaluable prior knowledge, the learning rate is dropped significantly to 5×10^{-5} with a linear decay schedule.

7.3 Computational Complexity and Hardware Optimization

The core bottleneck of the Transformer is the self-attention matrix multiplication. As noted by Vaswani et al., the computational complexity of self-attention per layer scales quadratically with sequence length: $O(n^2 \cdot d)$, whereas recurrent layers scale linearly: $O(n \cdot d^2)$. Because our sequence length n (1024) is large, the QK^T operation is highly memory intensive.

Table 1: Maximum Path Lengths and Per-Layer Complexity

Layer Type	**Complexity per Layer**	**Sequential Ops**	**Max Path Length**
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent (RNN)	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$

To execute training within strict memory limitations, optimizations are active:

- **Mixed Precision (FP16):** Model weights and activations are cast to 16-bit floats, halving VRAM requirements and activating hardware Tensor Cores.
- **Gradient Accumulation:** Simulating a larger batch size by running multiple forward/backward passes on micro-batches, accumulating gradients iteratively, and stepping the optimizer only after multiple cycles.

- **Context Truncation:** Due to the strict 1024-token limit, lengthy articles are truncated from the bottom up to ~ 800 tokens. Journalistic prose heavily utilizes the "inverted pyramid" structure, ensuring critical facts remain localized in the preserved lead paragraphs.

8 Inference: Autoregressive Decoding Strategies

Inference is the generative phase. The model is fed an unseen article appended with the `TL;DR:` trigger. Because the model outputs a vector of unnormalized logits (z_i) for all 50,257 tokens, decoding strategies are required to select the optimal text sequence.

8.1 Temperature Scaling

Prior to executing the softmax function, raw logits are divided by a constant Temperature variable T:

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \tag{15}$$

- $T = 1.0$**:** Represents the true, unmodified probability distribution.
- $T < 1.0$ **(e.g., 0.7):** Sharpens the distribution, suppressing less likely tokens. This makes the model more confident and factual, reducing hallucinations but increasing the risk of generic, repetitive phrasing.
- $T > 1.0$**:** Flattens the distribution, increasing sequence diversity and creative randomness.

8.2 Truncation Sampling: Top-K and Top-P

Pure Greedy decoding (always selecting $\arg\max p_i$) inevitably traps autoregressive models in infinite repetitive loops (e.g., "The the the..."). We apply stochastic sampling strategies to guarantee fluency:

- **Top-K Sampling:** The vocabulary is sorted by probability, and the selection pool is hard-capped to the top K tokens. The remainder are zeroed out.
- **Top-P (Nucleus) Sampling:** The model calculates the cumulative distribution function (CDF) of the sorted probabilities. It truncates the pool to the smallest set of tokens whose cumulative probability exceeds P (e.g., 0.92). This allows the candidate pool to dynamically expand or contract based on the model's mathematical confidence at any given step.

9 Evaluation Metrics and Expected Results

9.1 Quantitative Benchmarks (ROUGE)

The primary assessment employs the ROUGE (Recall-Oriented Understudy for Gisting Evaluation) metric suite to compare generated text against the human-written targets.

- **ROUGE-1:** Measures unigram (single word) overlap. Assesses basic factual content coverage and keyword extraction.
- **ROUGE-2:** Measures bigram (two-word phrase) overlap. Evaluates local grammatical fluency and phrase-level retention.
- **ROUGE-L:** Measures the Longest Common Subsequence (LCS). Assesses overall structural similarity and sentence-level syntactical flow, regardless of exact word adjacency.

9.2 Hypothesized Performance Delta

By maintaining identical data throughput and hardware constraints, the performance delta directly quantifies how much "general intelligence" contributes to "task-specific performance."

- **Pretrained Model:** Projected to achieve ROUGE-L scores > 30.0. Leveraging its generalized linguistic prior, it is expected to reliably synthesize multi-sentence, human-readable summaries that mimic standard news highlights.
- **Scratch Model:** Projected to achieve ROUGE-L scores < 15.0. The model will likely identify relevant nouns (keys) via attention matrices but fail to assemble them into grammatically valid sequences, frequently degenerating into fragmented outputs.

10 Discussion and Limitations

10.1 Linguistic Pathologies

1. **Hallucination:** Abstractive models are highly susceptible to generating plausible but factually incorrect text. This pathology manifests when the pre-trained weights favor a strong global association (e.g., associating "Microsoft" heavily with

"Bill Gates") that overrides the local contextual information provided in the source article.

2. **Degenerate Repetition:** Decoder-only architectures frequently fall into autoregressive looping failures. While Repetition Penalties ($\alpha = 1.2$) artificially suppress previously used tokens, looping remains a structural flaw in unidirectional pathfinding.

10.2 Hardware and Domain Restrictions

The 1024-token context window represents a hard ceiling. Articles exceeding this length suffer from information truncation, potentially losing concluding arguments or final statistics. Furthermore, because training is restricted entirely to news datasets, performance on specialized out-of-domain documentation (e.g., legal contracts or medical journals) is expected to degrade sharply without subsequent specialized fine-tuning.

11 Conclusion

This comparative implementation illustrates that while the Transformer's $O(1)$ sequential self-attention matrix mathematically enables infinite-range context tracking, the architectural framework alone is insufficient for high-quality abstractive generation without immense data scaling.

The *de novo* training protocol demonstrates that synthesizing complex grammatical syntax alongside task-specific condensation logic is heavily constrained by dataset scale and compute limits. Conversely, the pre-trained framework proves that a foundational model, possessing a rich internal representation of semantic structures learned via unsupervised Next Token Prediction, can be highly sample-efficient. When fine-tuned using targeted prefix logic (`TL;DR:`), the decoder-only architecture successfully adapts its vast linguistic prior to execute precise, high-fidelity abstractive summarization.

References

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems, 30. https://arxiv.org/abs/1706.03762

[2] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language Models are Unsupervised Multitask Learners*. OpenAI Blog.

[3] Hermann, K. M., Kocisky, T., Grefenstette, E., Espeholt, L., Kay, W., Suleyman, M., & Blunsom, P. (2015). *Teaching Machines to Read and Comprehend*. Advances in Neural Information Processing Systems, 28.

[4] Gowrishankar, P. *Newspaper Text Summarization (CNN/DailyMail)*. Kaggle Dataset. https://www.kaggle.com/datasets/gowrishankarp/newspaper-text-summarization-cnn-dailymail

[5] Wang, A., et al. (2024). *Transformer Explainer: Interactive Learning of Text-Generative Models*. Polo Club of Data Science, Georgia Institute of Technology. https://poloclub.github.io/transformer-explainer/

[6] Khandekar, S. (2023). *The Transformer: A Visual Exploration*. https://transformer.sujayk.me/

A Technical Appendices

A.1 Project Architecture and Codebase Structure

The system implementation relies entirely on the PyTorch ecosystem, integrated tightly with HuggingFace `transformers` and `accelerate` libraries to manage FP16 memory allocations on consumer GPU hardware.

- `data_engine/`: Contains Python modules responsible for BPE tokenizer instantiation, regex-based artifact removal, tensor sequence packing, and dynamic padding.
- `models/`: Defines the customized configuration classes for the 6-layer and 12-layer causal transformer variants, including custom overrides for Pre-LN routing.
- `trainers/`: Houses the main training loop mechanics, including cross-entropy loss masking over the source article and gradient accumulation algorithms.
- `eval/`: Contains the autoregressive generation scripts utilizing Nucleus (Top-P) sampling and the automated ROUGE scoring framework.

A.2 Algorithm: Autoregressive Inference Generation Loop

Below is the formalized pseudocode governing the generative decoding phase of the system. It details the exact sequence of logic required to parse source text, execute temperature scaling, and dynamically sample the token distributions.

Algorithm 1 Autoregressive Summarization Inference

Require: Source Article A, Max Generation Length M, Temperature T, Nucleus probability P

$S \leftarrow$ BPE_Tokenize("¡—startoftext—¿" $+ A +$ "\n\nTL;DR:\n")
$t \leftarrow 0$
while $t < M$ **do**
    $L \leftarrow$ ModelForwardPass(S) ▷ Yields logits for all vocabulary tokens
    $L_{last} \leftarrow L[-1]$ ▷ Extract logits corresponding to the final sequence position
    $L_{last} \leftarrow L_{last}/T$ ▷ Apply Temperature scaling to manipulate confidence
    $P_{dist} \leftarrow$ Softmax(L_{last}) ▷ Convert to probability distribution
    $P_{filtered} \leftarrow$ TopPSampling(P_{dist}, P) ▷ Apply Nucleus truncation
    $token_{next} \leftarrow$ SampleFrom($P_{filtered}$) ▷ Stochastic selection
    if $token_{next} ==$ "¡—endoftext—¿" **then**
        break ▷ Generation naturally terminates
    end if
    $S \leftarrow$ Append($S, token_{next}$) ▷ Autoregressively expand the context
    $t \leftarrow t + 1$
end while
return BPE_Decode(S) ▷ Convert integer sequence back to human-readable text